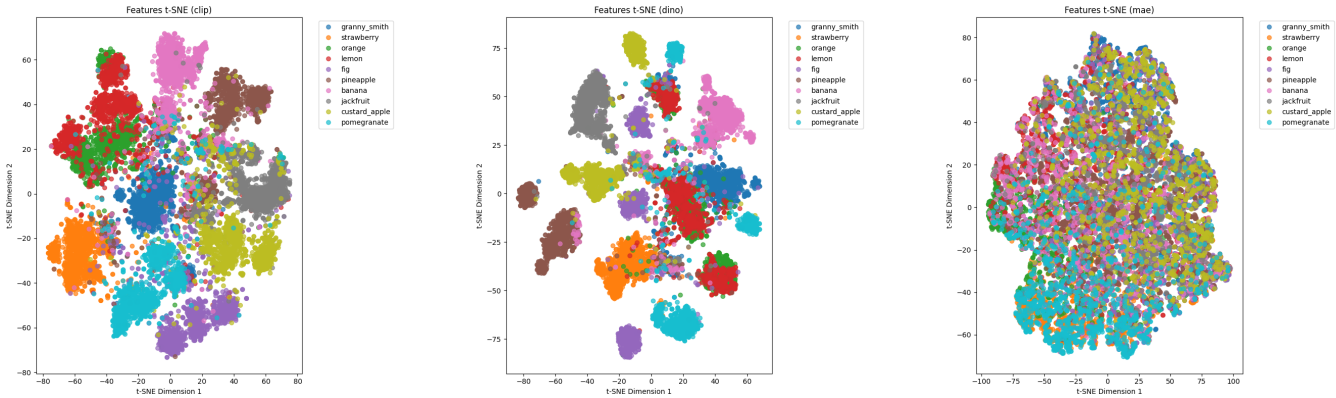
I. Part 1: Representation Learning

A. Feature Extraction and t-SNE Analysis



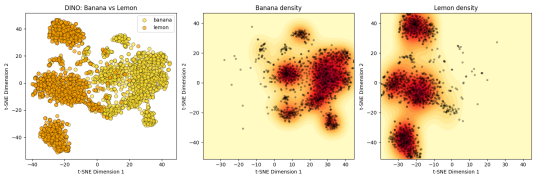
(a) CLIP Features t-SNE Plot (b) DINO Features t-SNE Plot (c) MAE Features t-SNE Plot

Fig. 1: t-SNE feature plots for CLIP, DINO, and MAE backbones

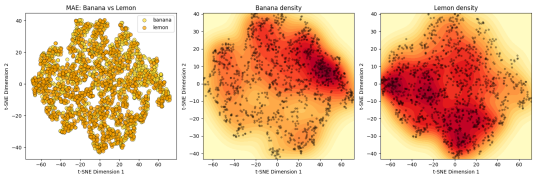
The three pre-training objectives produce clear differences in feature spaces when visualized with t-SNE. CLIP (Contrastive Language-Image Pre-training) learns a shared image-text embedding by bringing matching image–text pairs closer and pushing mismatched pairs apart. As seen in the extracted feature t-SNE plot Fig. 1 (a), CLIP has separated each class cluster visibly enough, indicating a meaningful understanding. Distinctly different classes, such as strawberries and bananas, are clustered very far apart. However, there is also a general uncertainty seen by the data points overlapping between clusters. This is most heavily seen between classes that share similar characteristics, such as orange and lemon, which heavily overlap with one another. This is due to CLIP aligning the images with the text for its representations, in an effort to keep matching text-image pairs together. It therefore generates meaningful embeddings containing both semantic and visual similarities, which is why the citrus fruits of lemon and orange are closely clustered.

DINO (DIstillation with NO labels) produces the strongest isolation of class clusters, as seen in Fig. 1 (b). There is also much less uncertainty in the cluster separability compared to CLIP, as seen by the decrease in mixing and overlap. Certain classes, such as jackfruit and pineapple, are much cleaner and isolated by DINO compared to CLIP. This is likely due to its features from its teacher-student network, with the teacher network parameters being updated as an exponential moving average of the student weights ($\theta_t = \lambda\theta_{t-1} + (1-\lambda)\theta_s$). This makes it learn through direct features and without relying on class labels or semantic priors. It therefore prioritizes learning through visual structure over language and semantic similarities, as in CLIP.

MAE (Masked Autoencoders) learns through reconstructions of randomly masked patches of the image. It therefore prioritizes pixel-level reconstruction over semantic similarities. MAE has the poorest performance, with a low separability of the class clusters, as seen in Fig. 1 (c). This is because the embeddings from reconstruction contain low level visual features such as textures and colors, which does not account for semantic similarities. One example is that bananas and lemons are heavily clustered together due to their yellow pixels encoding a similarity for MAE. Comparing how DINO and MAE cluster bananas and lemons, as seen in the 2D density contour plots in Fig. 2, MAE learns almost identical density distributions for the two fruits due to their colors, while DINO learns clear spatial separation and can distinguish the two classes based on higher-level features.



(a) DINO density plots for banana vs lemons



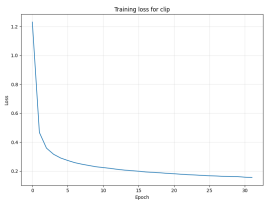
(b) MAE density plots for banana vs lemons

Fig. 2: Banana vs Lemon Density plots of DINO and MAE t-SNE projections

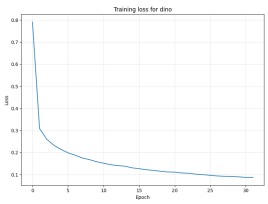
B. Linear Probing

Linear probing tests how linearly separable the pre-trained features are by freezing the backbone, extracting a fixed feature vector for every image, and training a single linear classifier on top. Because only the linear head is learned, the performance indicates the quality of the frozen representation space. CLIP had the highest validation accuracy for linear probing, at 92.4%, followed by DINO's 87.8% and then MAE at 68.6%. DINO starts with the lowest initial training loss at 0.8, as seen in Fig. 3 (b). It also converges to the lowest final loss at 0.09, which means that DINO benefits from the linear separability gained by freezing the features. CLIP also performs well, with a moderate initial loss of 1.2, but a low 0.1 final loss. Both CLIP and DINO show a steep decrease and consistent convergence, which means the features extracted during their run were already sufficient for good classification before freezing. MAE performs the worst, with the final loss of 0.9 being almost a magnitude higher than CLIP and DINO. This means the linear head had struggles finding a well-established decision boundary in MAE's feature space.

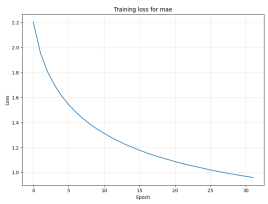
The ranking of models does generally follow the t-SNE performances. MAE's poor performance (68.6%) is also shown in its lack of class structure in its t-SNE visualization. However, although DINO's clusters in t-SNE were the most isolated, CLIP still achieves the highest accuracy for linear probing (92.4% vs 87.8%). This is because t-SNE and linear probing measure different properties of the embedding space. t-SNE focuses on local neighborhood structure, so although CLIP has more overlap in its clusters compared to DINO, it likely has better global structure and linear separability.



(a) CLIP training loss



(b) DINO training loss



(c) MAE training loss

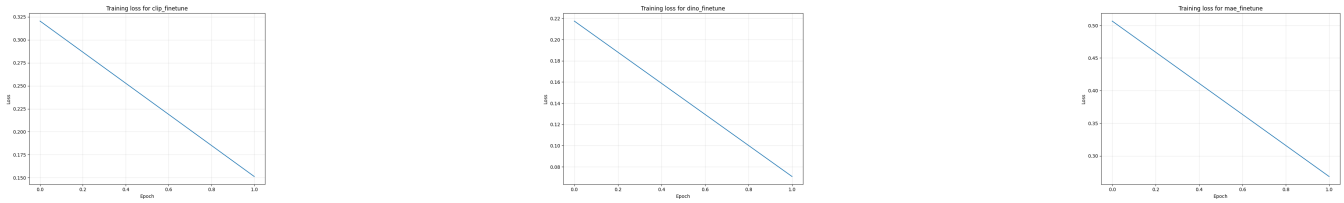
Fig. 3: Linear Probe Training loss curves for CLIP, DINO, and MAE

C. Fine-tuning

Fine-tuning tests how the representations can adapt to downstream tasks by attaching a new linear classification head to the backbone and training the network on the dataset. Unlike linear probing, the gradients are allowed to update both the backbone and the head. CLIP maintains the highest validation accuracy for fine-tuning at 90%, albeit having a higher accuracy for linear probing (92.4%). The slight drop after fine-tuning is likely due to mild overfitting and a degradation of the pre-trained features. This behavior is also reported in prior work at a larger scale, coined as catastrophic forgetting [1]. As CLIP is a large and well-pretrained model, it can lose some of its generalizable representations on this smaller downstream dataset. However, the decrease in performance is modest, meaning the features were still effective. Both DINO and MAE have higher accuracies after fine-tuning compared to linear probing, with DINO at 89.6% and MAE at 86.4%, which is almost a 17.8% increase. DINO

benefited from the fine-tuning, as its self-distillation focused on strong visual features, which meant the fine-tuning could help provide better semantic representations to make it better equipped for the fruit classification task, and less on general-purpose uses. MAE had a high improvement because its original method of training with low-level reconstruction features meant it was less-suited for classification. The fine-tuning therefore adjusted the goal, from a reconstruction model into a classifier.

As seen in Fig. 4, the training loss curves differ compared to linear probing's in that they show a monotonic decrease over the epoch, with no visible convergence but low final losses at around 0.3-0.07. DINO has the lowest final training loss at 0.07, despite CLIP having the highest validation accuracy. This may be due to DINO slightly overfitting to the training set, whereas CLIP shows better generalizable representations.



(a) CLIP training loss (b) DINO training loss (c) MAE training loss

Fig. 4: Fine-tune Training loss curves for CLIP, DINO, and MAE

II. Part 2: Diffusion Models

A. Forward and Reverse Diffusion

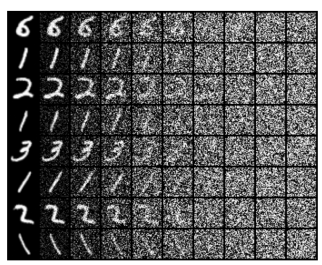


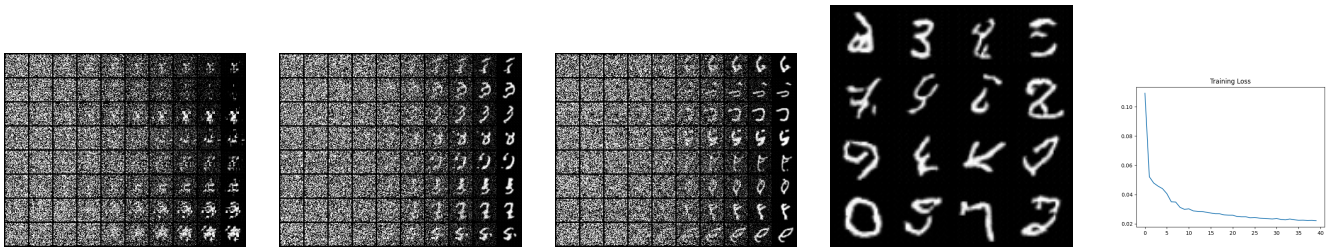
Fig. 5: Forward Unconditional Diffusion

The process of forward diffusion involves corrupting clean images into pure noise through adding Gaussian noise over t timesteps, as seen in Fig. 5 on the MNIST dataset. Reverse diffusion is the inverse, where the network learns to denoise the images and generate meaningful samples. The training and sampling follows the formulation from the Denoising Diffusion Probabilistic Models (DDPM) paper [2].

The denoising network uses the Diffusion Transformer (DiT) architecture [3]. The forward diffusion samples noisy images via $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$ and $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$. The network $\epsilon_\theta(x_t, t)$ is trained to predict noise using the DDPM objective $L = \mathbb{E}_{t,x_0,\epsilon}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$. The input images are patchified and embedded, and timesteps are encoded with sinusoidal embeddings. Then, at each diffusion timestep, the transformer blocks allow each image patch to attend to all other patches to capture the spatial structure of the digit. The output is then unpatchified to produce a per-pixel noise prediction. The reverse process is used for sampling: $x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right) + \sigma_t z$, where $z \sim \mathcal{N}(0, I)$. This iteratively denoises from the pure Gaussian noise images to generate new images.

This is done over 40 epochs to arrive at the samples seen in Fig. 6. For epoch 1 (a), the network is slowly starting to form faint outlines from the pure noise. It continues improving by epoch 16 (b), recognizing the structures of the digits but lacking in definition. By epoch 36 (c), the model is able to effectively learn the image from the Gaussian noise. This is proven by the training loss curve in Fig. 6 (e), which shows a sharp drop in

initial loss from 0.2 to 0.05, followed by convergence to approximately 0.022, meaning the DiT was able to quickly learn to predict the noise across all timesteps. The final generated samples in Fig. 6 (d) confirm this as they are sufficiently recognizable digits.



(a) Epoch 1 Denoise (b) Epoch 16 Denoise (c) Epoch 36 Denoise (d) Final samples (e) Loss

Fig. 6: Training evolution and loss convergence for the DiT model

B. Conditional Generation

The conditional model is an additional step from the unconditional case as it incorporates class labels. The embedding layer maps each class label c to a vector $e_c \in \mathbb{R}^d$. This class embedding is combined with the timestep embedding through $h = h_t + e_c$, where h_t is the sinusoidal timestep embedding passed through an MLP. This vector conditions every transformer block with adaptive LayerNorm (adaLN), so the network predicts class-conditional noise $\epsilon_\theta(x_t, t, c)$. The training objective becomes $L = \mathbb{E}_{t,x_0,\epsilon,c}[\|\epsilon - \epsilon_\theta(x_t, t, c)\|^2]$. During sampling, the desired class label is provided at each denoising step. This allows the reverse process to generate samples from the conditional distribution $p(x_0|c)$ and not the marginal $p(x_0)$.

As seen in the generated samples in Fig. 7 (b), the conditional model is able to clearly differentiate between classes and within, with each digit maintaining diversity with stroke and thickness. Compared to the samples from the unconditional model in Fig. 6 (d), which samples from the full mixture distribution, the conditional model provides explicit control over the generated output and therefore displays clearer digits.

The training loss curve, as shown in Fig. 7 (a), has similar trajectory to the one of the unconditional model. It has a sharp decrease from an initial loss of 0.2 to a convergence around 0.02. Despite this, the conditional model has better prediction performance, as seen by the generated samples. This is because the loss measures noise prediction accuracy, not output quality. The unconditional model must represent the joint distribution over all digit classes during denoising, which leads to averaged features. The conditional model knows the target class and can therefore make more focused predictions, which results in cleaner outputs.

An additional analysis is done on how the conditional model denoises. Attention maps were extracted from the first DiT block at timesteps $t = 490, 400, 250, 100, 10$ for a sampling of digit 7, shown in Fig. 8. When t is high ($t = 490$ to 400), the center patch focuses across the entire image as the model is still uncertain about the noise-dominated input. As the image denoises ($t = 250$ to 100), the center patch begins to localize its attention to the shape of the digit, as the model is beginning to better capture the spatial features. This progression in attention reflects the diffusion process, where early steps use the global spatial structure and later steps localize and refine the structural details.

(a) Loss curve (b) Generated samples

Fig. 7: Forward conditional diffusion: Training loss (left) and samples (right)

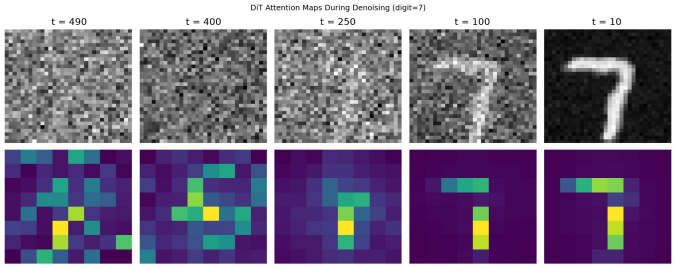
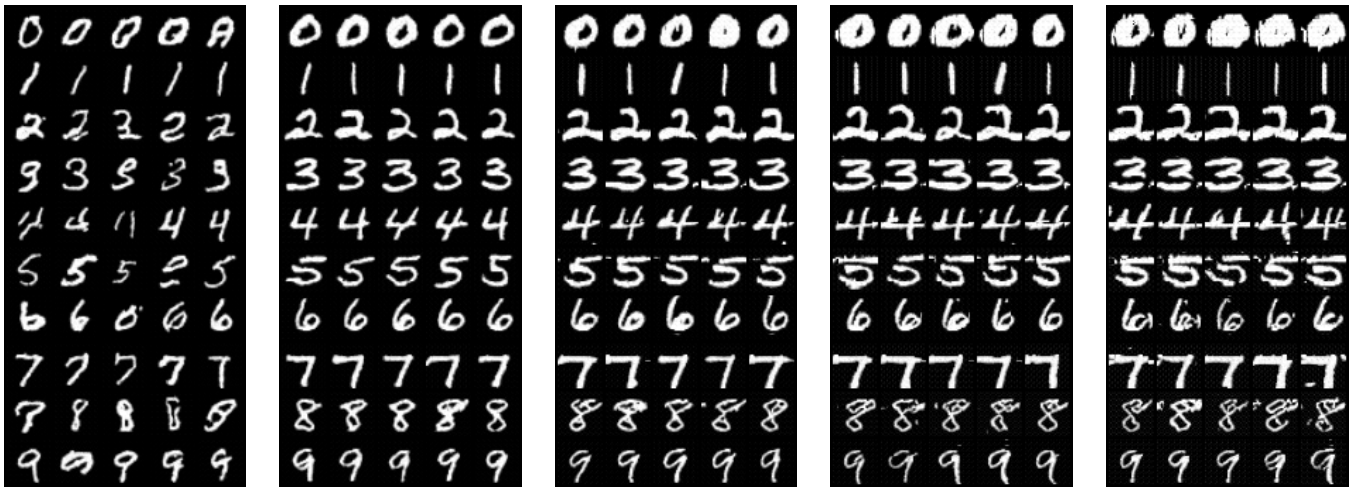


Fig. 8: Attention map visualization of digit 7 during denoising

C. Classifier-free Guidance

Classifier-free Guidance is a sampling method that linearly combines conditional and unconditional noise predictions, and is controlled by a guidance scale w. During training, class labels are randomly replaced with an unconditional class identifier with probability $p = 0.1$, so the network learns both conditional $\epsilon_\theta(x_t, t, c)$ and unconditional $\epsilon_\theta(x_t, t)$ denoising. At inference, two forward passes compute both predictions, which are combined as $\tilde{\epsilon} = \epsilon_\theta(x_t, t) + w\big(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t)\big)$ [4]. When $w = 1$, this is reduced to the standard conditional sampling. When $w > 1$, the model extrapolates away from the unconditional prediction and in turn amplifies the class features.

Various w values are tested, shown in Fig. 9. When w is 1, in Fig. 9 (a), the samples are only roughly accurate and contain inconsistencies with stroke thickness and shape. This is seen in its struggles with the digit four, as well as the top right digit zero sample resembling the letter A, losing its circular form. As w increases to 5 in Fig. 9 (b), and 10 in Fig. 9 (c), the model does a better job at giving sharper and uniform samples of each digit. For $w = 15$ and especially $w = 20$, the digits now have overly thick strokes, bleeding edges, and distorted pixels, which all degrade the quality of the samples. This is especially apparent in digits zero and four. This effect is because as the guidance scale increases, it begins to push samples farther away from the learned data distribution, which leads to the shown distortion. For the MNIST dataset, a moderate scale w of 5 or 10 is best suited.



(a) scale = 1 (b) scale = 5 (c) scale = 10 (d) scale = 15 (e) scale = 20

Fig. 9: Generated samples with difference guidance values (1, 5, 10, 15, 20)

References

[1] Reza Akbarian Bafghi, Nidhin Harilal, Claire Monteleoni, and Maziar Raissi. Parameter Efficient Fine-tuning of Self-supervised ViTs without Catastrophic Forgetting. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 3679-3684, 2024.

[2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. Advances in neural information processing systems, 33:6840–6851, 2020.
[3] William Peebles and Saining Xie. Scalable Diffusion Models with Transformers. In Proceedings of the IEEE/CVF international conference on computer vision, pages 4195–4205, 2023.
[4] Jonathan Ho and Tim Salimans. Classifier-free Diffusion Guidance. arXiv preprint arXiv:2207.12598, 2022.